Exp No: 4	**Convolutional Neural Network (CNN) from Scratch for Image Classification with Visualization of Learned Filters**

AIM:

To implement a Convolutional Neural Network from scratch to classify images, train it on a dataset of labeled images, evaluate its performance and visualize the filters learned in the convolution layer.

ALGORITHM / PROCEDURE:

1. Load the labeled image dataset (handwritten digits, 8x8 grayscale images, 10 classes), scale it to 0-1 and split it 80% / 20% into training and test sets.
2. **Convolution layer:** slide 8 filters of size 3x3 over the image (stride 1, no padding) to get eight 6x6 feature maps.
3. Apply the *ReLU* activation, then **max pooling** (2x2) to get eight 3x3 maps.
4. Flatten the maps (72 values) and pass them to a fully connected layer with 10 *softmax* outputs.
5. Compute the cross-entropy loss and apply **backpropagation** through the dense layer, the pooling layer (gradient goes to the max position) and the convolution layer.
6. Update all weights with the Adam optimizer (lr = 0.005, batch size 32) for 30 epochs.
7. Evaluate on the test set, then plot the learned 3x3 filters and the feature maps of a test image.

Fig 1: Architecture of the CNN

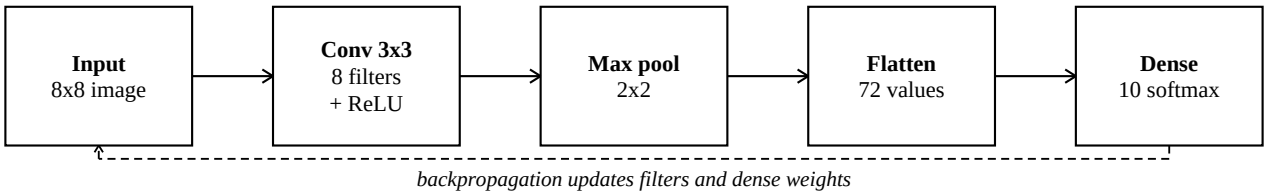
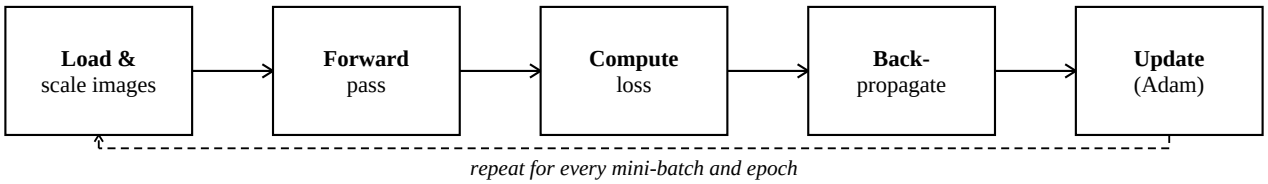


Fig 2: Training procedure



Commands / Settings used:

Layer	Output shape	Parameters
Input	8 x 8 x 1	0
Convolution (8 filters, 3x3) + ReLU	6 x 6 x 8	9 x 8 + 8 = 80
Max pooling (2x2)	3 x 3 x 8	0
Flatten	72	0
Dense + softmax	10	72 x 10 + 10 = 730
Total		**810**

PROGRAM:

```
import numpy as np
from numpy.lib.stride_tricks import sliding_window_view as swv
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

np.random.seed(1)
d = load_digits()
X = d.data.reshape(-1, 8, 8) / 16.0; y = d.target
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
Ytr = np.eye(10)[ytr]

# ---- CNN: Conv(8 filters 3x3) -> ReLU -> MaxPool(2x2) -> Flatten -> Dense(10) ----
F = 8
params = {"Wc": np.random.randn(9, F) * np.sqrt(2 / 9), "bc": np.zeros(F),
          "Wf": np.random.randn(3 * 3 * F, 10) * np.sqrt(2 / 72), "bf": np.zeros(10)}

def forward(X):
    N = len(X)
    cols = swv(X, (3, 3), axis=(1, 2)).reshape(N * 36, 9)          # im2col
    Z = (cols @ params["Wc"] + params["bc"]).reshape(N, 6, 6, F)    # convolution
    A = np.maximum(0, Z)                                            # ReLU
    Ar = A.reshape(N, 3, 2, 3, 2, F)
    Pool = Ar.max(axis=(2, 4))                                      # max pooling
    flat = Pool.reshape(N, -1)
    z = flat @ params["Wf"] + params["bf"]
    e = np.exp(z - z.max(1, keepdims=True))
    return dict(cols=cols, Z=Z, Ar=Ar, Pool=Pool, flat=flat, P=e / e.sum(1, keepdims=True))

def backward(c, Y):
    N = len(Y)
    dz = (c["P"] - Y) / N
    g = {"Wf": c["flat"].T @ dz, "bf": dz.sum(0)}
    dPool = (dz @ params["Wf"].T).reshape(N, 3, 3, F)
    mask = (c["Ar"] == c["Pool"][:, :, None, :, None, :])
    mask = mask / mask.sum(axis=(2, 4), keepdims=True)
    dA = (mask * dPool[:, :, None, :, None, :]).reshape(N, 6, 6, F)
    dZ = dA * (c["Z"] > 0)
    g["Wc"] = c["cols"].T @ dZ.reshape(-1, F); g["bc"] = dZ.sum(axis=(0, 1, 2))
    return g

m = {k: np.zeros_like(v) for k, v in params.items()}
v = {k: np.zeros_like(a) for k, a in params.items()}
t = 0
epochs, batch, lr = 30, 32, 0.005
for ep in range(1, epochs + 1):
    idx = np.random.permutation(len(Xtr))
    for s in range(0, len(idx), batch):
        b = idx[s:s + batch]; c = forward(Xtr[b]); g = backward(c, Ytr[b]); t += 1
        for k in params:                                            # Adam update
            m[k] = 0.9 * m[k] + 0.1 * g[k]; v[k] = 0.999 * v[k] + 0.001 * g[k] ** 2
```

```
            mh = m[k] / (1 - 0.9**t)
            vh = v[k] / (1 - 0.999**t)
            params[k] -= lr * mh / (np.sqrt(vh) + 1e-8)
    if ep % 5 == 0 or ep == 1:
        P = forward(Xtr)["P"]
        loss = -np.mean(np.sum(Ytr * np.log(P + 1e-12), 1))
        acc = np.mean(P.argmax(1) == ytr) * 100
        print(f"Epoch {ep:2d}/{epochs}  loss: {loss:.4f}  train accuracy: {acc:.2f}%")

P = forward(Xte)["P"]; pred = P.argmax(1)
print(f"\nTest loss     : {-np.mean(np.log(P[np.arange(len(yte)), yte] + 1e-12)):.4f}")
print(f"Test accuracy : {np.mean(pred == yte)*100:.2f}%")
print("Learned filters saved to filters.png, feature maps saved to feature_maps.png")

# ---------- Visualize learned filters ----------
W = params["Wc"].T.reshape(F, 3, 3)
fig, ax = plt.subplots(1, F, figsize=(10, 1.6))
for i in range(F):
    ax[i].imshow(W[i], cmap="gray")
    ax[i].set_title(f"Filter {i+1}", fontsize=8)
    ax[i].axis("off")
plt.tight_layout(); plt.savefig("img/filters.png", dpi=200); plt.close()

# ---------- Feature maps of one test image ----------
c = forward(Xte[:1]); A = np.maximum(0, c["Z"])[0]
fig, ax = plt.subplots(1, F + 1, figsize=(11, 1.6))
ax[0].imshow(Xte[0], cmap="gray")
ax[0].set_title(f"Input ({yte[0]})", fontsize=8)
ax[0].axis("off")
for i in range(F):
    ax[i + 1].imshow(A[:, :, i], cmap="viridis")
    ax[i + 1].set_title(f"Map {i+1}", fontsize=8)
    ax[i + 1].axis("off")
plt.tight_layout(); plt.savefig("img/feature_maps.png", dpi=200); plt.close()
```

OUTPUT:

Terminal - CNN from scratch

```
root@ip-10-10-5-10:~# python3 exp4.py
Epoch  1/30  loss: 1.5085  train accuracy: 72.16%
Epoch  5/30  loss: 0.2169  train accuracy: 94.99%
Epoch 10/30  loss: 0.0929  train accuracy: 98.61%
Epoch 15/30  loss: 0.0523  train accuracy: 99.23%
Epoch 20/30  loss: 0.0365  train accuracy: 99.51%
Epoch 25/30  loss: 0.0265  train accuracy: 99.65%
Epoch 30/30  loss: 0.0202  train accuracy: 99.72%

Test loss     : 0.0865
Test accuracy : 97.78%
Learned filters saved to filters.png, feature maps saved to feature_maps.png
```

Fig 3: Learned 3x3 convolution filters

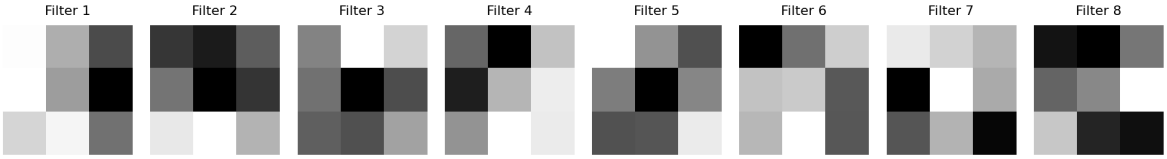
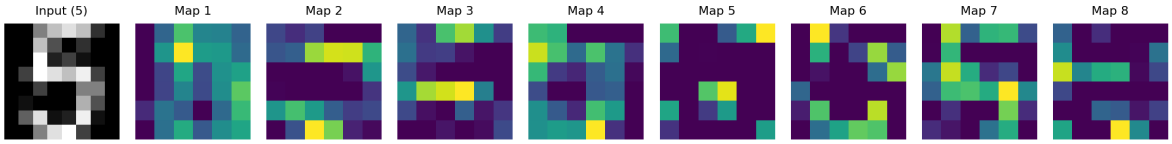


Fig 4: Input image and its feature maps after convolution + ReLU



RESULT:

Thus a Convolutional Neural Network was implemented from scratch using NumPy, with convolution, ReLU, max pooling and a dense softmax layer, and trained with backpropagation. The training loss fell from 1.5085 to 0.0202 over 30 epochs and the network reached a test accuracy of 97.78% (test loss 0.0865). The learned filters and feature maps were visualized.